

DCCO



Second Semester

hw21 UnitTests

OOP - 28434

Professor: Edison Lascano

T6 - Unit Tests

1.Patient Class

The screenshot shows an IDE window titled "ClinicManagement - Apache NetBeans IDE 27". The "Projects" pane on the left shows the project structure. The "Source" pane shows the code for `PatientTest.java` in the package `ec.edu.espe.clinicmanagementsystem.model`. The code includes imports for JUnit and AssertJ, and defines the `PatientTest` class with several test methods. The "Test Results" window at the bottom shows the results of running the tests. It indicates that 11 tests passed and 11 tests failed, all marked as "The test case is a prototype".

```
package ec.edu.espe.clinicmanagementsystem.model;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.Assertions.*;
import java.util.ArrayList;
import java.util.List;

/**
 *
 * @author César Vargas, Paradigm, BESPE
 */
public class PatientTest {

    public PatientTest() {
    }

    @BeforeAll
    public static void setUpClass() {
    }

    @AfterAll
    public static void tearDownClass() {
    }

    // Test methods
    @Test
    public void testSetPatientId() {
    }

    @Test
    public void testSetMedicalHistory() {
    }

    @Test
    public void testSetAddressValid() {
    }

    @Test
    public void testSetPayBill() {
    }

    @Test
    public void testSetAddressInvalid() {
    }

    @Test
    public void testSetPhoneInvalidLength() {
    }

    @Test
    public void testSetGenderInvalid() {
    }

    @Test
    public void testSetPhoneValid() {
    }

    @Test
    public void testSetFullNameValid() {
    }

    @Test
    public void testViewMedicalHistory() {
    }

    @Test
    public void testAddMedicalRecord() {
    }

    @Test
    public void testSetGenderValid() {
    }

    @Test
    public void testRequestAppointment() {
    }

    @Test
    public void testSetFullNameInvalid() {
    }
}
```

Test Results: 3 tests passed, 11 tests failed. (0.05 s)

- testSetPatientId Failed: The test case is a prototype.
- testSetMedicalHistory Failed: The test case is a prototype.
- testSetAddressValid Failed: The test case is a prototype.
- testPayBill Failed: The test case is a prototype.
- testSetAddressInvalid Failed: The test case is a prototype.
- testSetPhoneInvalidLength passed (0.002 s)
- testSetGenderInvalid passed (0.001 s)
- testSetPhoneValid Failed: The test case is a prototype.
- testSetFullNameValid Failed: The test case is a prototype.
- testViewMedicalHistory Failed: The test case is a prototype.
- testAddMedicalRecord Failed: The test case is a prototype.
- testSetGenderValid Failed: The test case is a prototype.
- testRequestAppointment Failed: The test case is a prototype.
- testSetFullNameInvalid passed (0.001 s)

Test Results Summary:

- setPhone Valid
- setFullName Valid
- viewMedicalHistory
--- Mostrando Historial Medico para Test User ---
No hay registros medicos disponibles.
- addMedicalRecord
- setGender Valid
- requestAppointment
Paciente Test User esta solicitando una cita.
- setFullName Invalid (Contains Numbers)

2.Doctor Class

The screenshot shows an IDE window titled "ClinicManagement - Apache NetBeans IDE 27". The "Projects" pane on the left shows the project structure. The "Source" pane shows the code for `DoctorTest.java` in the package `ec.edu.espe.clinicmanagementsystem.model`. The code includes imports for JUnit and AssertJ, and defines the `DoctorTest` class with several test methods. The "Test Results" window at the bottom shows the results of running the tests. It indicates that 10 tests passed and 10 tests failed, all marked as "The test case is a prototype".

```
package ec.edu.espe.clinicmanagementsystem.model;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.Assertions.*;
import java.util.Date;

public class DoctorTest {

    @Test
    public void testCreatePrescription() {
        System.out.println("Creando Prescripcion");
        Doctor instance = new Doctor();
        Patient p = new Patient();
        p.setPatientId(1);
        Prescription res = instance.createPrescription(p, 1, "Med", "Dose", new Date());
        assertEquals(res);
        fail("The test case is a prototype.");
    }

    @Test
    public void testDiagnose() {
        System.out.println("diagnose");
        Doctor instance = new Doctor();
        instance.diagnose(new Patient(), "Sano");
        fail("The test case is a prototype.");
    }

    @Test
    public void testCreatePrescriptionCheckDefaultInstructions() {
        System.out.println("createPrescription (Check Default Instructions)");
        Patient patient = new Patient();
        patient.setPatientId(1);
    }
}
```

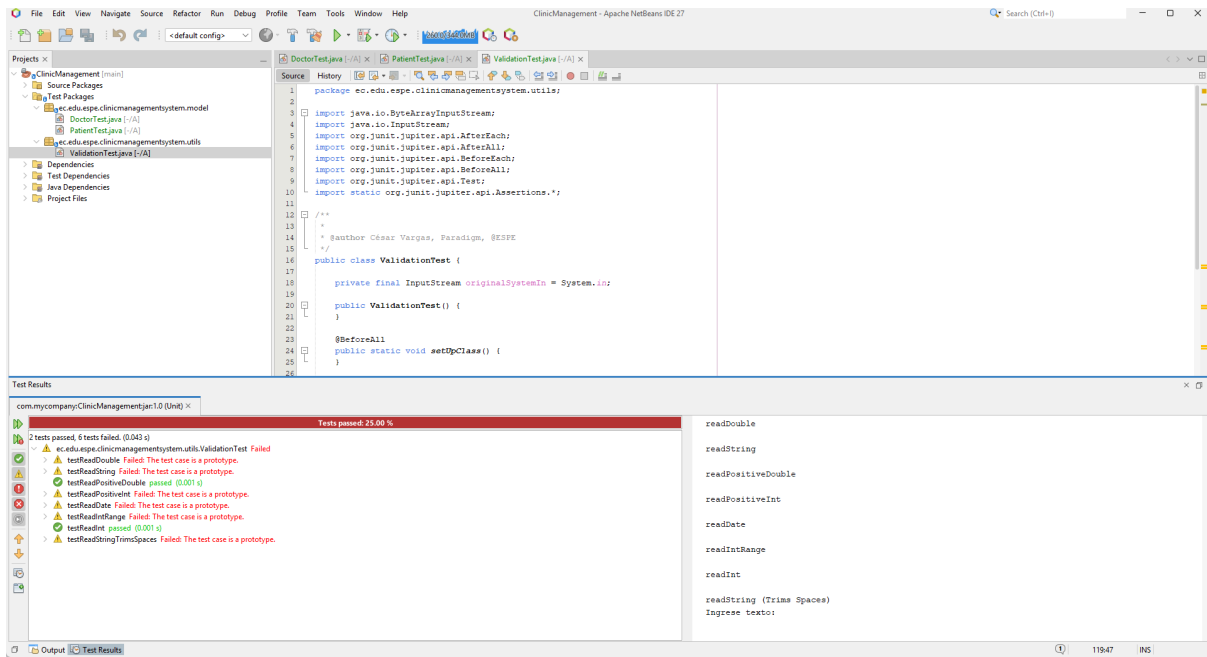
Test Results: 2 tests passed, 10 tests failed. (0.055 s)

- testSetDoctorId Failed: The test case is a prototype.
- testSetPassword passed (0.0 s)
- testSetFullName Failed: The test case is a prototype.
- testUpdateMedicalHistory Failed: The test case is a prototype.
- testViewAppointments Failed: The test case is a prototype.
- testCreatePrescriptionCheckDefaultInstructions Failed: The test case is a prototype.
- testDiagnose Failed: The test case is a prototype.
- testCreatePrescription Failed: The test case is a prototype.
- testSetSpecialty Failed: The test case is a prototype.
- testSetSpecialty Failed: The test case is a prototype.
- testSetUsername passed (0.001 s)
- testSetEmail Failed: The test case is a prototype.
- testSetPhone Failed: The test case is a prototype.

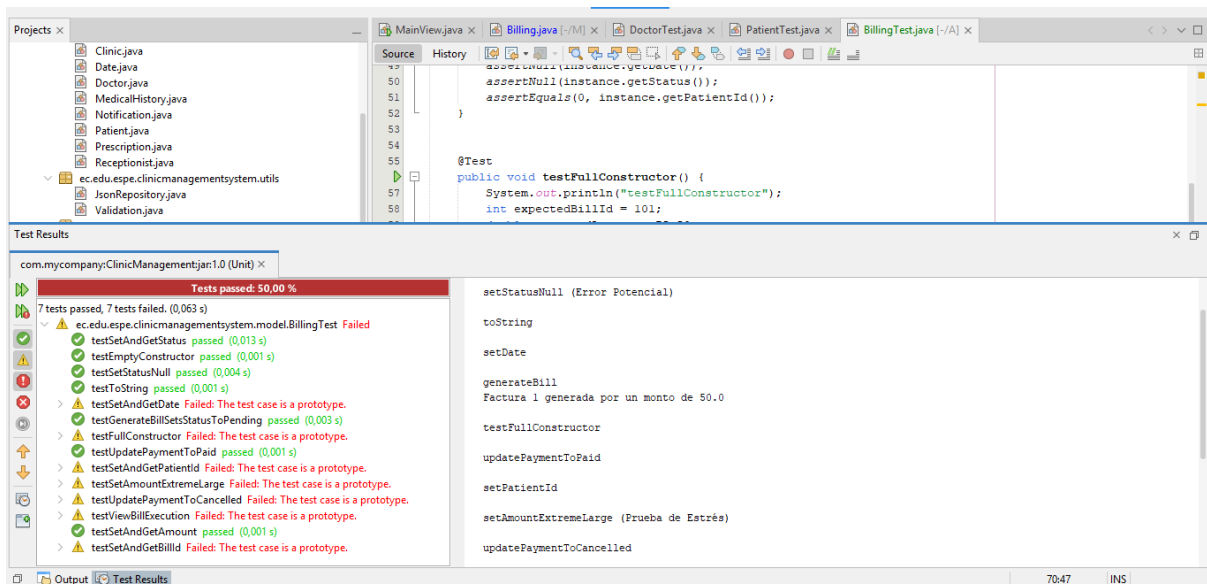
Test Results Summary:

- createPrescription (Check Default Instructions)
Dr. Dr. Test creando prescripcion para Paciente Test
- diagnose
null diagnostico a null con: Sano
- createPrescription
Dr. null creando prescripcion para null
- setSpecialty
- setUsername
- setEmail
- setPhone

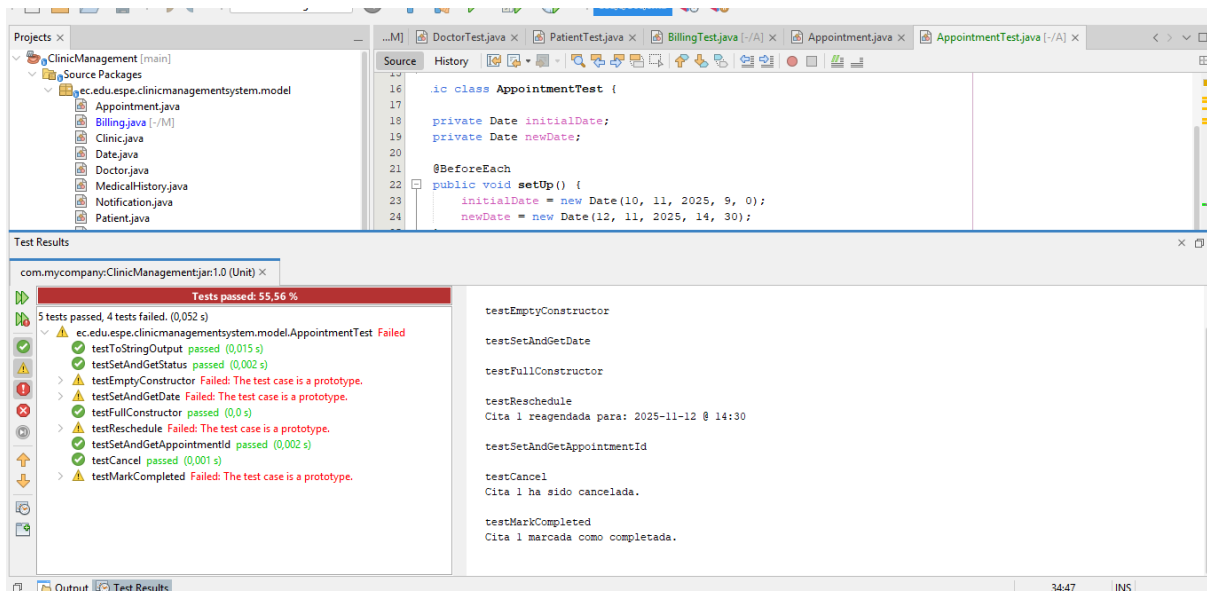
3.Validation Class



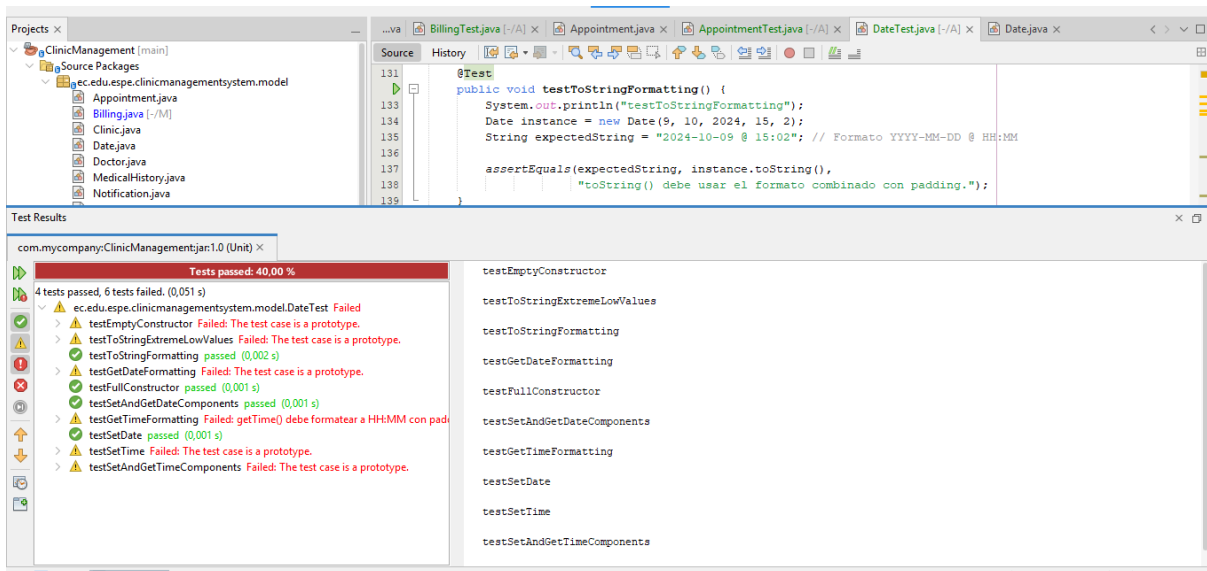
5.Biling Class



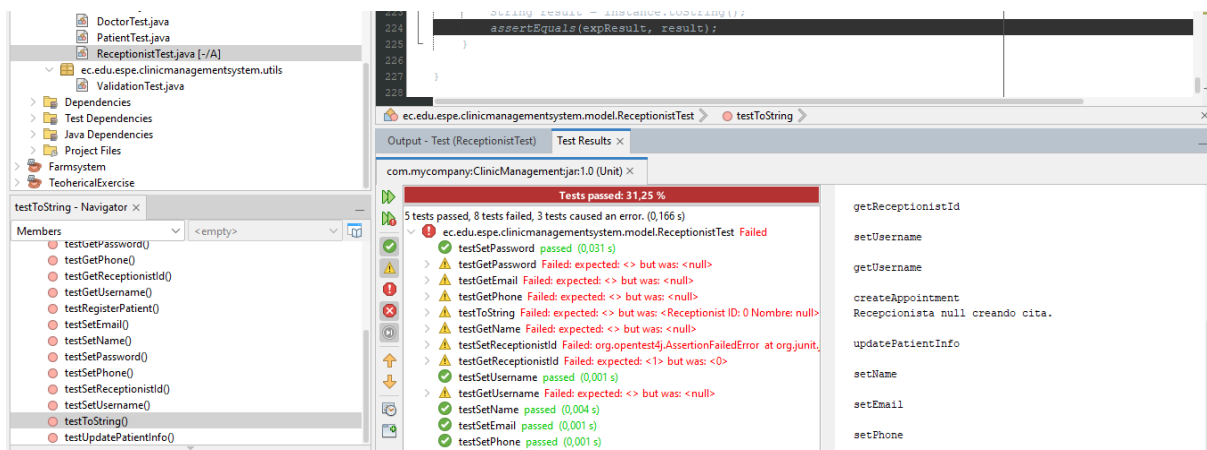
6.Appoiment Class



7. Date Class



8. Receptionist Class



9. Prescription Class

The screenshot displays an IDE with the `PrescriptionTest.java` file open. The left sidebar shows the project structure, including `ec.edu.espe.clinicmanagementsystem.model` and `ec.edu.espe.clinicmanagementsystem.utils`. The main editor shows the `PrescriptionTest` class with various test methods. The right sidebar shows the test results for `com.mycompany.ClinicManagementjar:1.0 (Unit)`. The results indicate that 5 tests passed and 8 tests failed, with a total of 1 test causing an error. The failed tests are:

- `testGetInstructions` Failed: expected: <> but was: <null>
- `testSetPatientId` Failed: The test case is a prototype.
- `testSetInstructions` passed (0,002 s)
- `testToString` Failed: expected: <> but was: <Prescription[prescriptionId=0, patientId=0, date=0, instructions=0, medication=0, prescriptionId=0, patientId=0, date=0, instructions=0, medication=0]>
- `testGetMedication` Failed: expected: <> but was: <null>
- `testGetDate` passed (0,002 s)
- `testSetMedication` passed (0,002 s)
- `testSetPatientId` Failed: The test case is a prototype.
- `testSetPrescriptionId` Failed: The test case is a prototype.
- `testSetDosage` passed (0,001 s)
- `testGetDosage` Failed: expected: <> but was: <null>
- `testSetDate` passed (0,003 s)
- `testGetPrescriptionId` Failed: The test case is a prototype.

The right sidebar also shows the `setMedication` method and its output, which includes the text: `--- Imprimiendo Prescripcion ---`, `ID Prescripcion: 0`, `Paciente ID: 0`, `setDate`, and `getPrescriptionId`.

10. Notification Class

The screenshot displays an IDE with the `NotificationTest.java` file open. The left sidebar shows the project structure, including `DoctorTest.java`, `NotificationTest.java`, `PatientTest.java`, `PrescriptionTest.java`, and `ReceptionistTest.java`. The main editor shows the `NotificationTest` class with various test methods. The right sidebar shows the test results for `com.mycompany.ClinicManagementjar:1.0 (Unit)`. The results indicate that 1 test passed and 4 tests failed, with a total of 1 test causing an error. The failed tests are:

- `testToString` Failed: expected: <> but was: <Notification{message=null, date=0}>
- `testGetDate` Failed: The test case is a prototype.
- `testGetMessage` Failed: expected: <> but was: <null>
- `testSetDate` Failed: The test case is a prototype.

The right sidebar also shows the `setMessage` method and its output, which includes the text: `setMessage`, `toString`, `getDate`, `getMessage`, and `setDate`.